Transactional Outbox

Outbox · Atomically save order + outbox row, relay later · Build with Waqar

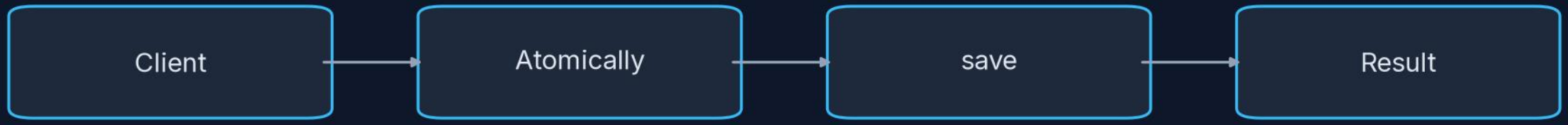
Today's Hook

- Problem: Atomically save order + outbox row, relay later
- Pattern: Outbox — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Atomically save order + outbox row, relay later
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Outbox: Atomically save order + outbox row, relay later

Real-World Scenario

- Running example: Atomically save order + outbox row, relay later
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.enterprise.outbox;

import com.buildwithwaqar.patterns.enterprise.outbox.Outbox;
/** Problem: never lose domain events when the broker is down — persist then relay. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Outbox problem: reliable publish ===");
    var o = new Outbox();
    o.placeOrder("ord-1", "{\"sku\":\"course\"}");
    System.out.println("unpublished=" + o.unpublished().size());
    o.publishDue();
    System.out.println("afterRelay=" + o.unpublished().size() + " orders=" + o.orders());
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.enterprise.outbox;

import java.util.ArrayList;
import java.util.List;

record OutboxMessage(String id, String topic, String payload, boolean published) {
    OutboxMessage markPublished() { return new OutboxMessage(id, topic, payload, true); }
}

/** Transactional outbox — persist business row + message atomically, publish async. */
public final class Outbox {
    private final List<String> orders = new ArrayList<>();
    private final List<OutboxMessage> messages = new ArrayList<>();

    public synchronized void placeOrder(String orderId, String payload) {
        orders.add(orderId);
        messages.add(new OutboxMessage(orderId, "orders", payload, false));
    }

    public synchronized List<OutboxMessage> unpublished() {
        return messages.stream().filter(m -> !m.published()).toList();
    }
```

Solution (2)

```
    public synchronized void publishDue() {
        for (int i = 0; i < messages.size(); i++) {
            var m = messages.get(i);
            if (!m.published()) messages.set(i, m.markPublished());
        }
    }

    public List<String> orders() { return List.copyOf(orders); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.enterprise.outbox;

import com.buildwithwaqar.patterns.enterprise.outbox.Outbox;
/** Problem: never lose domain events when the broker is down — persist then relay. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Outbox problem: reliable publish ===");
    var o = new Outbox();
    o.placeOrder("ord-1", "{\"sku\":\"course\"}");
    System.out.println("unpublished=" + o.unpublished().size());
    o.publishDue();
    System.out.println("afterRelay=" + o.unpublished().size() + " orders=" + o.orders());
  }
}
```

Walkthrough Output

```
=== Outbox problem: reliable publish ===
unpublished=1
afterRelay=0 orders=[ord-1]
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Outbox.

Related Patterns

- Contrast Outbox with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Outbox → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.